

FoodSloth

Food-Delivery Platform on Spring Boot Microservices

Complete Technical Documentation - services, calculations, RabbitMQ eventing,
synchronization, security and how everything is implemented

Java 21 · Spring Boot 3 · Spring Cloud Gateway · Eureka · RabbitMQ · MongoDB · Stripe · Cloudinary

Contents

1. Platform overview and the two hard rules
2. Architecture at a glance
3. Service-by-service guide
4. Inter-service communication with RabbitMQ
5. Event catalogue on food.exchange
6. The order pipeline and failure paths
7. Calculations and formulas
8. State machines
9. Synchronization and consistency
10. Security model
11. Data storage map
12. Friendly identifiers (user codes and order numbers)
13. Frontend implementation
14. Running the platform

1. Platform overview and the two hard rules

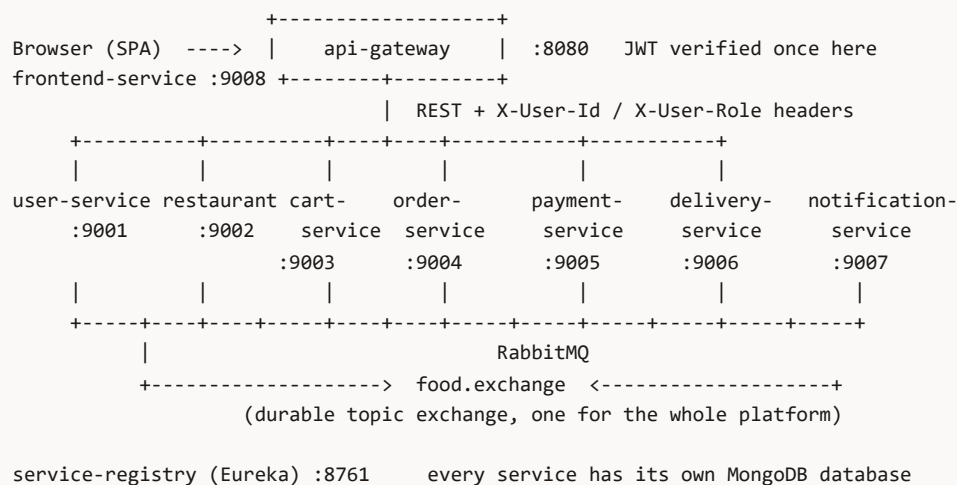
FoodSloth is a complete food-delivery platform: a customer browses the menu, checks out, pays (card via Stripe or cash on delivery), watches the kitchen cook, tracks the rider on a live map and receives push notifications - while an admin runs the kitchen screen, menu, rider dispatch, user management and the payment ledger from the same web app, and delivery men work a shift-based rider dashboard.

The whole platform follows two hard rules:

- **Client to service: REST only, always through api-gateway (port 8080).** The browser never talks to a backend service directly.
- **Service to service: RabbitMQ events only.** No service ever calls another service over HTTP. All cross-service state travels as messages on one shared durable topic exchange.

Technology stack: Java 21, Spring Boot 3, Spring Cloud Gateway, Eureka service discovery, Spring AMQP (RabbitMQ), Spring Data MongoDB (one Atlas database per service), Stripe Checkout + webhooks, Cloudinary signed uploads, and a build-free vanilla HTML/CSS/JS single-page frontend.

2. Architecture at a glance



Every backend service registers with Eureka, owns exactly one MongoDB database, and speaks to the rest of the platform exclusively through the shared exchange. A service that is down simply leaves messages waiting in its durable queue - nothing is lost and nothing else blocks.

3. Service-by-service guide

3.1 service-registry (port 8761)

Eureka server. No business logic, no database. Every other service registers with it and discovers peers through it. The gateway routes to services by Eureka service id, so no hard-coded host/port exists anywhere.

3.2 api-gateway (port 8080)

The only door into the platform for clients. Spring Cloud Gateway with:

- **Route forwarding** by path prefix (`/users/**` , `/restaurant/**` , `/carts/**` , `/orders/**` , `/payments/**` , `/deliveries/**` , `/notifications/**`) to the owning service via Eureka.
- **JWT verification once.** The gateway validates the token issued by user-service, then *strips any client-supplied* `X-User-Id` / `X-User-Role` headers and re-adds them from the verified claims. Downstream services trust these headers and never parse tokens themselves - spoofing the headers from outside is impossible.
- **CORS** for the frontend origin `http://localhost:9008` .
- A small public route set without a token: login/signup, the menu, the Stripe webhook and the Stripe return pages.

3.3 user-service (port 9001, DB food_user_db)

Owns accounts and identity.

- Signup/login for roles `CUSTOMER` , `ADMIN` , `DELIVERYMAN` ; issues the signed JWT every request carries. `ADMIN` signup additionally requires the `ADMIN_SIGNUP_KEY` secret (default `ad123`).
- Profile (`GET/PUT /users/me`), saved delivery addresses (CRUD + set-default), preferences (food preferences, dietary tags, default payment method).
- Admin user management: `GET /users` lists everyone; `DELETE /users/{id}` may delete customers and delivery men only - deleting an admin (including yourself) is rejected with 400/403.
- **Friendly IDs:** every user gets a sequential code with a role prefix - `A-n` (admin), `C-n` (customer), `D-n` (delivery man) - minted from atomic Mongo counters; existing users are backfilled lazily on login.
- **Publishes** `user.registered` for every new account and re-publishes it for every admin at startup (self-healing replay so notification-service always knows the admins). It consumes nothing and calls nothing.

3.4 restaurant-service (port 9002, DB food_restaurant_db)

Single-tenant: the whole service is one restaurant. It plays two roles - menu owner and pricing/kitchen engine.

REST side

- Profile, open/closed switch (`PATCH /restaurant/status`), operating hours.
- Menu CRUD with a per-item availability flag; signed Cloudinary upload parameters so the browser uploads photos directly to Cloudinary.
- Kitchen screen API: active queue oldest-first, queue by status, single ticket with line items and totals, and staff actions `accept` , `reject` (with reason), `ready` .

Event side

- Consumes `cart.checked-out` and `order.reorder-requested` on one shared queue `restaurant.order-intake.queue` (same shape, same handling: price against today's menu).
- Publishes `restaurant.order-priced` or `restaurant.order-unavailable` (unknown/sold-out item, closed restaurant, empty cart).
- On `order.confirmed` the ticket enters the kitchen queue (status `QUEUED`); staff decisions become `restaurant.order-accepted` , `restaurant.order-rejected` , `restaurant.order-ready` .
- Consumes `order.cancelled` to stop cooking.

This service is the **only place a price is decided** - see section 7.1.

3.5 cart-service (port 9003, DB food_cart_db)

One cart per user: add item, update quantity, remove, clear, checkout. The path carries the cart owner and must match the authenticated user. Checkout is the entry point of the whole pipeline and is deliberately fire-and-forget:

1. Mints the UUID `orderId` - the platform's correlation key.

2. Publishes `cart.checked-out` (items, drop address + coordinates, payment method, note).
3. Returns **202 with the orderId immediately**; pricing and order creation happen downstream over the broker, so checkout stays instant even if restaurant- or order-service is down.

3.6 order-service (port 9004, DB food_order_db)

The state machine of the platform (section 8.1). There is **no endpoint to create an order**: orders come into existence when `restaurant.order-priced` arrives. REST is read/action only: `GET /orders/{id}` (404 simply means pricing hasn't finished), `GET /orders/me`, `PATCH /orders/{id}/cancel` (allowed until cooking starts; triggers refund downstream), `POST /orders/{id}/reorder` (202 + fresh orderId, re-priced at today's prices).

Consumes: `restaurant.order-priced`, `restaurant.order-unavailable`, `restaurant.order-accepted`, `restaurant.order-rejected`, `restaurant.order-ready`, `payment.succeeded`, `payment.failed`, `delivery.started`, `delivery.completed`.

Publishes: `order.payment-requested`, `order.confirmed`, `order.cancelled`, `order.delivered`, `order.reorder-requested`.

Every order also gets a sequential **orderNo** (#1, #2, ...) from a Mongo counter; it is carried inside every published event so restaurant, payment, delivery and notification can all show the friendly number without ever calling order-service.

3.7 payment-service (port 9005, DB food_payment_db)

Owens all money movement through Stripe; the only service touching the outside payment world. There is **no endpoint to start a payment** - payments start exclusively from `order.payment-requested`, so nobody can be charged for an order that does not exist.

- Card orders get a Stripe Checkout session; COD orders auto-succeed immediately.
- Amounts stored in minor units (poisha); the UI divides by 100 for display.
- `payment.succeeded` / `payment.failed` result from the signature-verified Stripe webhook, the checkout return pages, or the COD shortcut.
- Consuming `order.cancelled` automatically refunds any succeeded card charge - the platform's compensation mechanism instead of a distributed transaction.
- `POST /payments/order/{orderId}/retry` re-issues a declined/expired session; admin ledger `GET /payments`, `POST /payments/{id}/refund`.

3.8 delivery-service (port 9006, DB food_delivery_db)

Rider app on one side, customer tracking feed on the other, admin dispatch on top.

- **Shift**: `POST /deliveries/riders/online` (captures GPS once), `POST /deliveries/riders/offline` (refused while a job is open), `GET /deliveries/riders/me`.
- **Capacity**: each rider gets `delivery.slots-per-day` (5) slots per calendar day; the counter resets with the date. Assignable only while online with a free slot.
- **Assignment**: a Delivery record is created when `restaurant.order-ready` arrives (status `PENDING_ASSIGNMENT`). The admin assigns it to any online rider with free slots (`POST /deliveries/{id}/assign`); the job is mandatory for the rider. Consuming `order.cancelled` releases the rider.
- **Admin-only cancellation**: `PATCH /deliveries/{id}/cancel` (role ADMIN only) cancels an assigned/started delivery and **frees the rider's slot**; delivered deliveries cannot be cancelled (refund through Payments instead). Riders have no cancel capability anywhere.
- **Rider flow**: `accept` (publishes `delivery.started` - the moment the order turns `OUT_FOR_DELIVERY`), `picked-up`, `delivered` (publishes `delivery.completed`).

- **Tracking:** GET `/deliveries/order/{orderId}/track` returns rider position, remaining distance and ETA; restricted to the parcel owner, the rider, or an admin.

ETA/distance math and the GPS sentinel guard are in section 7.3.

3.9 notification-service (port 9007, DB `food_notification_db`)

The platform's biggest listener and a pure consumer - it publishes nothing. Push is the only delivery channel. It learns who exists from `user.registered` (including the admin replay), listens to the full order lifecycle and money events, stores every message in a per-user inbox, and exposes inbox REST (`/notifications/me` , unread count, mark read/read-all, device token registration) that backs the frontend's notification page. All queues are durable, so it can restart or lag without dropping a single message.

3.10 frontend-service (port 9008)

Serves the single-page web app from `src/main/resources/static` - plain HTML/CSS/JS, no build step, no database, not on the event bus. The browser talks straight to api-gateway with the JWT. One codebase, three role-based experiences (customer, admin/kitchen, rider). See section 13.

4. Inter-service communication with RabbitMQ

4.1 The rules

1. No service ever calls another service over HTTP. REST exists only for clients (through the gateway) and for Stripe talking to payment-service.
2. Each service owns its own MongoDB database. If a service needs another service's data it listens to that service's events and keeps its own copy (restaurant keeps kitchen copies of orders, delivery keeps delivery copies, notification keeps an inbox copy of everything).
3. No shared event JAR. Each service owns a minimal record for the events it consumes; the contract is the routing key plus JSON field names. Consumer records carry `@JsonIgnoreProperties(ignoreUnknown = true)` and nullable fields, so a publisher adding a field (like `orderNo`) never breaks any consumer.

4.2 Topology

- One shared durable topic exchange: `food.exchange` .
- Routing keys: `<owning-service>.<event>` - the publisher is always obvious from the key (e.g. `payment.succeeded` , `restaurant.order-ready`).
- Queues: `<consuming-service>.<event>.queue` , declared **only by the consumer**. Publishers declare only the exchange; identical re-declaration is a no-op in AMQP, so services start in any order.
- Reliability: `default-requeue-rejected: false` (poison messages are dropped after retries instead of looping forever) and `retry.enabled: true`, `max-attempts: 3` .
- All queues durable: if a consumer is down, messages wait for it.

4.3 Why events instead of REST/RPC between services

Temporal decoupling: a down consumer's messages wait in durable queues, so nothing cascades. Synchronous REST is used only where waiting is mandatory and the waiter is the *client* (menu browse, order polling, tracking, Stripe checkoutUrl). RabbitMQ RPC is deliberately never used - it is still synchronous and would rebuild HTTP badly.

5. Event catalogue on food.exchange

Routing key	Publisher	Consumed by	Meaning
user.registered	user	notification	New account; replayed for every admin at user-service startup
cart.checked-out	cart	restaurant	Cart checked out; price these items
order.reorder-requested	order	restaurant	Past order reordered (same shape as checkout)
restaurant.order-priced	restaurant	order	Checkout priced; creates the order
restaurant.order-unavailable	restaurant	order	Checkout could not be priced
restaurant.order-accepted	restaurant	order, notification	Kitchen accepted the order
restaurant.order-rejected	restaurant	order, notification	Kitchen rejected the order
restaurant.order-ready	restaurant	order, delivery, notification	Food packed; needs a rider
order.payment-requested	order	payment	Charge this order (amount in minor units)
payment.succeeded	payment	order, notification	Charge succeeded (COD auto-succeeds)
payment.failed	payment	order, notification	Charge failed
order.confirmed	order	restaurant, notification	Money secured; kitchen may cook
order.cancelled	order	restaurant, payment, delivery, notification	Order off; stop cooking, refund, release rider
delivery.assigned	delivery	notification	Rider assigned to the delivery
delivery.started	delivery	order, notification	Rider accepted; order goes OUT_FOR_DELIVERY
delivery.completed	delivery	order	Food handed over
order.delivered	order	notification	Final stop of the pipeline

6. The order pipeline and failure paths

```
checkout (202 + orderId)
-> cart.checked-out          restaurant prices it against the live menu
-> restaurant.order-priced   order-service creates the order, asks for payment
-> order.payment-requested    payment charges the card / auto-succeeds COD
-> payment.succeeded         order.confirmed -> kitchen queue -> accept -> cook
-> restaurant.order-ready    admin assigns a free rider (5 slots/rider/day)
-> delivery.started           rider accepts - customer sees OUT_FOR_DELIVERY
-> delivery.completed         order.delivered
```

Failure paths:

- `restaurant.order-unavailable` closes the checkout before any order exists.

- `payment.failed` lets the customer retry with a fresh Stripe link.
- `restaurant.order-rejected` and customer cancels raise `order.cancelled`, which stops cooking, releases any rider (slot freed) and refunds any card charge.
- Admin delivery cancel (`PATCH /deliveries/{id}/cancel`) frees the rider's slot without touching the order lifecycle.

Correlation: `orderId` is a UUID minted by cart-service at checkout and carried through every event of that order. The client gets it in the 202 and follows it everywhere (`GET /orders/{orderId}` , `GET /payments/order/{orderId}` , `GET /deliveries/order/{orderId}/track` , `GET /notifications/order/{orderId}`).

7. Calculations and formulas

7.1 Pricing (restaurant-service, the only place prices are decided)

Configuration: `restaurant.delivery-fee: 40.0` (flag-fall/base fee), `restaurant.delivery-fee-per-km: 12.0` , `restaurant.delivery-fee-max: 300.0` , `restaurant.tax-rate: 0.05` , road factor 1.3 (constant).

```
lineTotal = round2(unitPrice × quantity)
itemsTotal = round2(Σ lineTotal)
roadKm = haversineKm(restaurant, drop) × 1.3
deliveryFee = min(300, round2(40 + roadKm × 12)) | 40 flat if either side has no
coordinates
tax = round2(itemsTotal × 0.05)
grandTotal = round2(itemsTotal + deliveryFee + tax)
```

`haversineKm` is the great-circle distance with earth radius 6371 km; the 1.3 factor stretches straight-line distance because roads are never straight. If the restaurant or the drop address has no coordinates (null or 0,0), the flat base fee of 40 applies so an unpinned address still gets a sensible price. `round2` keeps two decimals so the total always equals the sum of the lines shown. Pricing also validates: empty cart, closed restaurant, unknown item, sold-out item, invalid quantity - any of these raises `restaurant.order-unavailable` .

7.2 Money units (payment-service)

Amounts travel and are stored in **minor units (poisha)**: the priced grand total in taka is multiplied by 100 before Stripe. The UI and the admin ledger divide by 100 for display. Stripe Checkout sessions carry the exact integer amount; the webhook is signature-verified before any state change.

7.3 Distance and ETA (delivery-service)

Configuration: `delivery.average-speed-kmph: 22` , `delivery.handover-minutes: 5` , `delivery.assignment-radius-km: 8` , `delivery.location-max-age-minutes: 5` , `delivery.slots-per-day: 5` .

```
distanceKm(from, to) = haversine great-circle, R = 6371 km
etaMinutes(km) = ceil(km / 22 × 60) + 5
At ASSIGN: estimate = distance(rider fix, pickup) + distance(pickup, drop) (two legs)
At PICK_UP: estimate = distance(rider fix, drop) (restaurant leg done)
```

Without a believable rider fix the first leg is measured from the restaurant itself.

GPS sentinel guard: browsers/old clients can report the (0,0) area or the ± 0.0001 sentinel - a point in the ocean that would poison every distance and ETA (e.g. 10,000 km / 27,000 min). `GeoPoint.isReal()` treats any fix with

$|\text{lat}| < 0.01$ and $|\text{lng}| < 0.01$ as "no fix": going online stores null, auto-assign skips such riders, estimates fall back to the restaurant, and the frontend maps start from the restaurant instead of the ocean.

Assignment eligibility: rider status AVAILABLE, a free slot today, a real location newer than 5 minutes, and within 8 km of the pickup; nearest first. Slot counters reset when the calendar date changes; a cancelled job refunds the slot, a completed one keeps it.

7.4 Sequential friendly identifiers (atomic Mongo counters)

```
counter doc: { _id: "user-code-C" | "user-code-D" | "user-code-A" | "order-no", seq: n }
mint = mongoTemplate.findAndModify(query(_id), update.inc("seq", 1),
upsert(true).returnNew(true), "counters") -> prefix + "-" + seq
```

The atomic find-and-modify increment guarantees uniqueness under concurrency and numbers never repeat after deletion. Users: A-n / C-n / D-n by role, backfilled lazily on login for pre-existing accounts. Orders: sequential orderNo shown as #n, carried inside every published event so all services and all notification texts can display it without calling order-service.

8. State machines

8.1 Order (order-service)

PENDING_PAYMENT → CONFIRMED → PREPARING → READY → OUT_FOR_DELIVERY → DELIVERED, with PAYMENT_FAILED, REJECTED and CANCELLED as terminal side states. Transitions are driven purely by events: order-priced creates PENDING_PAYMENT; payment.succeeded → CONFIRMED; payment.failed → PAYMENT_FAILED; order-accepted → PREPARING; order-rejected → REJECTED; order-ready → READY; delivery.started → OUT_FOR_DELIVERY; delivery.completed → DELIVERED (+ publishes order.delivered); cancel/reject → CANCELLED (+ order.cancelled).

8.2 Kitchen ticket (restaurant-service)

AWAITING_PAYMENT (priced, not yet paid - hidden from the kitchen screen) → QUEUED (paid) → ACCEPTED (cooking) → READY (packed), with REJECTED/CANCELLED side states.

8.3 Delivery and rider (delivery-service)

Delivery: PENDING_ASSIGNMENT → ASSIGNED → ACCEPTED → PICKED_UP → DELIVERED, plus CANCELLED (order cancel or admin cancel).

Rider: OFFLINE / AVAILABLE / ON_DELIVERY; going offline is refused while a job is open.

8.4 Payment (payment-service)

PENDING → SUCCEEDED | FAILED; SUCCEEDED → REFUNDED on cancel compensation or admin refund; CANCELLED for charges voided by order cancellation.

9. Synchronization and consistency

The system is **eventually consistent by design**; synchronization happens on three levels.

9.1 Service-to-service: durable events

When an order changes state, the owning service publishes an event; every interested service has a durable queue bound to that key and applies the change to its own copy whenever able. A down consumer accumulates messages and catches up on restart - the producer never waits, never retries synchronously, never fails because a peer is offline. Example - one cancellation ripples through four services:

```
order-service publishes order.cancelled
-> restaurant-service: kitchen ticket closed, cooking stopped
-> payment-service:    card charge refunded if it existed
-> delivery-service:   assigned rider released, slot freed
-> notification-service: customer and admins informed
```

9.2 The self-healing admin replay

Notification-service must know the admins, but user-service never exposes an internal sync API. Solution: user-service re-publishes `user.registered` for every admin at startup. An admin registered before the event existed, or a message lost while the broker was down, is repaired by the next restart; signup itself never fails because of the broker (publish is best-effort).

9.3 Client-to-service: REST polling behind 202

Because checkout returns 202 before the order exists, clients poll deliberately (no websockets): checkout polls `GET /orders/{id}` until it stops 404ing; the kitchen screen polls its queue; the rider dashboard polls its current job; the tracking map polls the track endpoint every few seconds and draws the marker itself; the inbox polls the unread count for the badge. Polling is safe because every list endpoint reads the service's own database, kept fresh by the listeners above.

9.4 Consistency choices

- No distributed transactions - compensation instead: `order.cancelled` triggers refund, rider release and kitchen stop as independent, idempotent reactions.
- Listeners are idempotent on the order id, so at-least-once redelivery cannot double-create.
- Optimistic client UX: the UI moves forward on local state and reconciles on the next poll.

10. Security model

- **JWT once at the edge.** user-service signs; api-gateway verifies, strips and re-issues `X-User-Id` / `X-User-Role` from verified claims; services trust headers only (they are unreachable from outside anyway).
- **Role enforcement** at both gateway routes and service controllers (e.g. admin-only delivery cancel checks `X-User-Role`; cart paths must match the authenticated user; tracking is limited to owner/rider/admin).
- **Admin signup key** (`ADMIN_SIGNUP_KEY` , default `ad123`) gates the ADMIN role at signup.
- **Stripe webhook signature verification** before any payment state change.
- **Secrets** are `${ENV_VAR:default}` placeholders only (`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `JWT_SECRET`, `ADMIN_SIGNUP_KEY`) supplied via `.env`, `secrets.yml` or the IDE; real keys are never committed.
- **Deletion protection:** admins cannot delete admins (including themselves); customers and delivery men can be deleted.

11. Data storage map

Service	Database	Key collections
user-service	food_user_db	users, addresses (embedded), counters (user codes)
restaurant-service	food_restaurant_db	restaurant profile + menu, kitchen_orders
cart-service	food_cart_db	carts (one per user)
order-service	food_order_db	orders, counters (order-no)
payment-service	food_payment_db	payments
delivery-service	food_delivery_db	deliveries, riders
notification-service	food_notification_db	notifications (per-user inbox), devices, known users

No shared collections and no cross-database joins anywhere.

12. Friendly identifiers (user codes and order numbers)

Raw UUIDs are kept as the internal correlation keys, but every user-facing surface shows short sequential identifiers: users as `A-n / C-n / D-n` (admin / customer / delivery man) in the admin users table (with All / Customers / Delivery men / Admins tabs), the profile page and confirmations; orders as `#n` across the customer order list and detail, kitchen tickets, rider dashboard, live tracking, the admin payment ledger and every notification text ("Order #12 ..."). Old records without a number fall back to a UUID prefix so nothing ever renders blank.

13. Frontend implementation

- Vanilla-JS SPA served by frontend-service on 9008; hash routing; views self-register via `App.register(path, { roles, render })`; DOM built with a tiny `UI.el(tag, attrs, ...children)` helper; inline SVG icon library (`Icon.of`) - no emoji anywhere.
- `api.js` attaches the JWT and calls the gateway at :8080; role-gated navigation; session cache kept in sync because every user-service mutation returns the fresh user.
- Live behaviour is polling: order detail 3s, rider job 5s, tracking map 5s, kitchen queue and unread badge a few seconds.
- Maps are Google embeds: the rider job page always shows restaurant → customer; the customer track page starts at the rider's live GPS when real, else the restaurant.
- Cache-busting: every asset carries a `?v=N` suffix bumped after edits, plus `no-store` cache-control headers from the service.
- Role experiences: customer (hero + menu, food search, cart, checkout, Stripe, orders + live tracking, inbox, profile), admin (kitchen screen with order details and admin-only delivery cancel, menu CRUD, dispatch, users with role tabs and protected delete, payment ledger with refunds), rider (shift toggle with slot bar, mandatory accept, pickup/deliver flow, history).

14. Running the platform

Service	Port	MongoDB database	Gateway path
service-registry	8761	-	-
api-gateway	8080	-	(entry point)
user-service	9001	food_user_db	/users/**
restaurant-service	9002	food_restaurant_db	/restaurant/**
cart-service	9003	food_cart_db	/carts/**
order-service	9004	food_order_db	/orders/**
payment-service	9005	food_payment_db	/payments/**
delivery-service	9006	food_delivery_db	/deliveries/**
notification-service	9007	food_notification_db	/notifications/**
frontend-service	9008	-	(serves the web UI)

Start order: RabbitMQ and MongoDB first, then service-registry, then the seven services in any order, then api-gateway, then frontend-service (open <http://localhost:9008>). Because queues and the exchange are durable and re-declaration is a no-op, any other start order still converges.

Secrets: STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET (payment-service), JWT_SECRET (gateway + user-service, must match), ADMIN_SIGNUP_KEY (user-service). Supply via .env, secrets.yml or IDE environment variables.

End of document.